Pyics - Data-Oriented Calendar Automation System

Show Image

Show Image

Show Image

Show Image

Show Image

Show Image

Maintainer: OBINexus Computing
Engineering Lead: Nnamdi Okpala
Status: Active Development — Phase 3.1.6.3 Single-Pass Architecture 🚧
License: MIT

Overview

Pyics is a modular Python package for **automated calendar generation**, milestone tracking, and secure .ics

distribution across distributed systems. Built on a **Data-Oriented Programming (DOP)** foundation with **lambda calculus function composition**, Pyics enforces zero-trust principles through pure transformations, immutable state, and cost-aware single-pass module loading.

Core Philosophy

- **Pure functions** handle all calendar transformations — no side effects, no mutation
- **Immutable state** flows through composable pipelines via lambda calculus primitives
- **Single-pass loading** enforces strict dependency ordering, eliminating circular imports
- **Cost-aware architecture** assigns priority indices and compute weights to each domain for boot-time optimization
- **Policy hooks** provide pre/post processing with zero business logic bleed

---

Problem Statement

Current `.ics` solutions suffer from monolithic coupling, insufficient automation for compliance tracking, and poor distributed systems integration. Pyics solves this through:

- Policy-wrapped calendar logic with hooks and escalation chains
- Version-isolated architecture (`v1/`, `v2/`, `v3-preview/`) with CLI discovery
- Authenticated `.ics` sharing with email domain validation and API tokens
- Civil/legal milestone tracking with automated compliance deadline management
- Audit trail generation for regulatory and institutional accountability

---

Architecture

Single-Pass Domain Loading

Pyics enforces a strict single-pass load order across nine core domains. Each domain declares its `priority_index`, `compute_time_weight`, and `exposure_type` at registration. No domain may import from a domain with a higher load order.

```
Load Order:
  primitives(10) → protocols(20) → structures(30) → composition(40)
  → validators(50) → transformations(60) → registry(70) → routing(80) → safety(90)
```

Performance targets: boot < 100ms | memory < 50MB | abstraction overhead < 5%

Core Domain Reference

Domain	Load Order	Priority	Exposure	Compute Weight	Status
`primitives`	10	1	core_internal	0.1	Active
`protocols`	20	1	version_required	0.05	Active
`structures`	30	2	version_required	0.2	Active
`composition`	40	2	version_required	0.3	Active
`validators`	50	3	version_required	0.4	Consolidated
`transformations`	60	3	version_required	0.6	Consolidated
`registry`	70	4	version_required	0.5	Active
`routing`	80	4	version_required	0.7	Active
`safety`	90	5	core_internal	0.3	Active

Consolidated domains: `core/validation` merged into `validators`; `core/transforms` and `core/logic` eliminated and their functionality absorbed into `transformations` and `composition` respectively.

Directory Structure

```
pyics/
├── core/
│   ├── primitives/         # Atomic operations, zero dependencies, O(1)
│   │   ├── data_types.py
│   │   ├── operations.py
│   │   ├── relations.py
│   │   ├── config.py
│   │   ├── __init__.py
│   │   └── README.md
│   ├── protocols/          # Type-safety contracts, interface definitions
│   ├── structures/         # Immutable calendar data containers
│   ├── composition/        # Lambda calculus composition engine
│   ├── validators/         # Consolidated validation (merged from validation/)
│   ├── transformations/    # Pure transformation library (absorbs legacy transforms/)
│   │   └── legacy_transforms/
│   ├── registry/           # Thread-safe component registration and discovery
│   ├── routing/            # Pipeline execution coordination
│   ├── safety/             # Thread-safety utilities and concurrency guards
│   └── ioc_registry.py     # Single-pass IoC registry with cost-aware loading
├── v1/                     # Stable LTS implementation
```

```
├── v2/                    # Enhanced features + telemetry
├── v3-preview/            # Experimental AI/blockchain features
├── audit/                 # Version-agnostic business audit logic
├── calendar/              # Core calendar factories
├── notify/                # Notification engines
├── cli/
│   ├── __init__.py
│   └── main.py            # Unified CLI entry point (IoC-registered commands)
├── tests/
│   ├── unit/
│   ├── integration/
│   ├── e2e/
│   ├── fixtures/
│   └── conftest.py
└── __init__.py
```

DOP Foundation

The `core/` layer enforces four compliance requirements for all operations:

1. **Zero Trust** — all operations route through registered transforms in the IoC registry
2. **Purity** — functions must not mutate state or produce side effects
3. **Immutability** — data structures are frozen after creation
4. **Composition** — complex operations use lambda calculus primitives exclusively

```python
from pyics.core.composition import Lambda

# Left-to-right pipeline construction
pipeline = Lambda.pipe(
    transform.validate_events,
    transform.apply_timezone,
    transform.generate_ics,
)

result = pipeline(input_data)  # Immutable state flows through
```

```python
from pyics.core.structures import ImmutableEvent

event = ImmutableEvent(summary="Review", dtstart=..., dtend=...)
updated = event.with_metadata(location="Room A")
# Original event unchanged
```

Installation

```bash
pip install pyics
```

```bash
# From source
git clone https://github.com/obinexusmk2/pyics.git
cd pyics
pip install -e .
pip install -e ".[dev]"
pip install -e ".[enterprise]"
```

Quick Start

```bash
pyics setup                                  # Initialize IoC registry
pyics version                                # Verify installation
pyics domain status                          # Check all domain load states
pyics validate-architecture                  # Confirm single-pass compliance
pyics generate --template milestone --output out.ics # Generate calendar
pyics-repl                                   # Interactive REPL
```

```python
from pyics.core.primitives import create_entity, AtomicValue
from pyics.core.protocols import ValidationProtocol
from pyics.core.structures import create_event_structure
from datetime import datetime

event = create_event_structure(
    event_id="meeting_001",
    title="Compliance Review",
    start_time=datetime(2025, 3, 1, 9, 0),
    end_time=datetime(2025, 3, 1, 10, 0)
)
```

CLI Reference

```bash
# Architecture
pyics validate-architecture
pyics domain status [domain_name]
pyics domain load-order
pyics domain metadata primitives --format json

# Calendar operations
pyics generate      # Generate .ics files
pyics audit         # Audit existing calendars
pyics distribute    # Distribute calendar files
pyics notify        # Send notifications
pyics verify        # Verify calendar integrity

# Version-pinned execution
pyics-v1 <command>
pyics-v2 <command>
```

Use Cases

Civil/Legal Milestone Tracking — Automated compliance deadline management with penalty escalation chains. **Enterprise Operations** — Team-wide milestone automation with authenticated `.ics` distribution. **Research Workflows** — Academic project timeline calendarization with audit trails. **Compliance Systems** — Policy-enforced `.ics` generation with complete audit logging.

Development

```bash
```

```
# Test suite
pytest tests/unit/
pytest tests/integration/
pytest tests/e2e/
pytest tests/migration/
python scripts/testing/benchmark.py

# Architecture validation
python -c "
from pyics.core.ioc_registry import get_registry, validate_architecture
print('Architecture valid:', validate_architecture())
print('Load order:', get_registry().get_load_order())
"
```

Contribution Guidelines

1. Fork and branch from `main`
2. Add modules under the correct domain directory following the standard 6-file pattern (`data_types.py`, `operations.py`, `relations.py`, `config.py`, `__init__.py`, `README.md`)
3. Register transforms via `@register_transform` — never call cross-domain functions directly
4. Maintain 90%+ test coverage with type hints throughout
5. Validate with `pyics validate-architecture` before submitting a PR

Roadmap

v1.0.0 (Current) — Single-pass architecture, consolidated core domains, DOP foundation, CLI IoC container, milestone tracking, SMTP distribution.

v2.1 — WebSocket real-time sync, advanced policy engine, REST API, telemetry and audit trail completion.

v2.2 — Performance optimization for large datasets, custom validation rule engine.

v3.0 Preview — AI-driven smart scheduling, blockchain audit trails, ML prediction models.

License

MIT License — Copyright © 2025 OBINexus Computing

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is provided to do so, subject to the conditions stated in the full MIT license text.

Links

- **Repository**: github.com/obinexusmk2/pyics
- **Issue Tracker**: github.com/obinexusmk2/pyics/issues
- **Changelog**: CHANGELOG.md
- **Architecture Spec**: docs/DOP_FOUNDATION.md

Built by the OBINexus Engineering Team — *When systems fail, we build our own.*